Trabalho 01 - Estrutura de dados II
TAD Grafo e algoritmos - Relatório

Rayane Kaylla Venâncio Gomes Alves - 0117243
Mateus Otacílio Silva Gomes - 0117242

Introdução

Desenvolvido em *C*, este projeto consiste em um tipo abstrato de dados (TAD) para grafos e alguns dos algoritmos essenciais para operação em grafos, que visa facilitar a modelagem e melhorar a compreensão dessa poderosa ferramenta da computação. Para facilitar a codificação da estrutura e dos algoritmos, foi utilizado TADs básicas de Fila, Lista e Pilha, que operam sob tipos genéricos de dados - ponteiro *void*.

TAD *Grafo*

A *struct* grafo possui um número estático de vértices e arestas. Nela temos: número máximo de vértices, número máximo de arestas, e duas *structs,* que são ponteiros pois se comportam como vetores, para vértices e arestas. Para fins de simplificação na manipulação dos índices, a posição 0 dos vetores de vértices e arestas foram designados para metadados.

***struct* vertice:** uma posição para inteiros e outra para uma lista. Os vértices nessa estrutura são armazenados por uma lista de incidência. Há um controle de vértices livres, que ainda não foram criados, por um vetor auxiliar de inteiros.
Metadados - *lista*: número de vértices criados (uma lista com um único *node)*.

***struct* aresta:** uma posição de inteiro para *omega* e outra para *alfa*, que são, respectivamente, vértice de saída e entrada de uma aresta.
Metadados - *alfa*: número de arestas criados;
omega: controla a próxima posição livre para criação de aresta.

Algumas considerações de implementação:

Salvar em .txt: Optou-se por salvar o arquivo texto em um formato que fosse possível ser facilmente convertido para o tipo .dot, utilizando o *software Graphviz,* que nos foi apresentado durante a disciplina de Matemática Discreta. Com ele, podemos analisar grafos e digrafos visualmente. Também foi criada uma função extra: *GBexportaDot*, que visa facilitar a exportação entre digrafo e grafo dentro do arquivo .dot.

Carregar em .txt: Após a inclusão das funções extras de *removeVertice* e *removeAresta*, a função de carregar grafos precisou de uma refatoração lógica, pois agora nada garante a ordem sequencial dos vértices, podendo existir lacunas. Agora o carregador converte os vértices existentes em uma sequência de IDs correta e ajusta as extremidades das arestas para esses novos IDs. Isso permite salvar e carregar o grafo sem referências inválidas.

Removendo vértices: Para a correta execução da remoção de um vértice dentro do grafo, todas as suas referências têm que ser deletadas nas listas dos outros vértices. Como a TAD lista é para manipulação de dados genéricos, optou-se pela criação de uma função auxiliar *removeValorDaLista*, que basicamente abre a lista em questão, localiza um inteiro, que é o ID daquele vértice, e remove o valor que havia sido alocado por um *malloc.* Assim garantimos a correta liberação, sem lixo de memória.

Incremento e decremento dentro da lista: Como dito anteriormente, a lista na posição 0 do vetor de vértices possui apenas um elemento, que armazena o número de vértices que foram efetivamente criados. Como estamos lidando com uma lista para guardar tipos genéricos, criamos duas funções para facilitar o incremento e o decremento do contador da lista: *incrementaContadorVertices* e *decrementaContadorVertice*.

A maior parte das funções foram implementadas com relativa facilidade, sendo necessário apenas algumas linhas de código. As funções relacionadas aos vértices do grafo foram um pouco mais custosas, pois, na lista de incidência, os vértices são armazenados por um número **positivo** (quando são de entrada) e **negativo** (quando são de saída). Dessa forma, foi necessário considerar o valor absoluto para executar a lógica corretamente.

Algoritmos essencias

Como já sabíamos, mais ou menos, o funcionamento dos outros algoritmos e a importância dos algoritmos de busca para a execução deles, os algoritmos de busca foram desenvolvidos primeiro. Aqui estão na ordem em que foram implementados:

BEL: A busca em largura foi desenvolvida com base no algoritmo fornecido pelo professor, de Cormen, que utiliza um esquema de cores para separar os vértices que já foram visitados, em processamento, e os já finalizados. Com a TAD grafo, percorrer todos os vértices foi incrivelmente fácil, bastando apenas um for com as funções de *GAprimaSaida* e *GAproxSaida.* A lista também foi utilizada.

BEP: A busca em largura foi desenvolvida com base no algoritmo recursivo de Cormen, também fornecido pelo professor. Optou-se pela criação do algoritmo recursivo pela sua facilidade e funcionamento, porém bastaria trocar a lista por uma pilha no algoritmo BEL para transformá-lo em um BEP. Também, para facilitar a codificação, os vetores de controle foram alocados como variáveis globais, tornando o código mais enxuto, mesmo não sendo boa prática o uso de variáveis globais.

CC: Para o algoritmo para componentes conexos, bastou eliminar o vértice de início da BEP, que é fornecido pelo usuário, pelo primeiro vértice do grafo, novamente o uso da TAD facilitou essa tarefa, onde todos os vértices do grafo são analisados, e quando um outro vértice não foi analisado na recursão da BEP, então há outro componente conexo.

AGM: O algoritmo da árvore geradora mínima foi implementado com base em Cormen, com o algoritmos de Prim, a utilização da nossa TAD Grafo facilitou significativamente a implementação. As funções de iteração do grafo e a lista de incidência permitiram identificar e acessar eficientemente as arestas de menor peso, garantindo a construção da árvore com menor peso e sem a formação de ciclos indesejados nas conexões.

CMC: O algoritmo de caminho mais curto visa encontrar a rota de menor custo de um vértice de origem para os demais. Com base no algoritmo de Dijkstra, a lógica de execução tira proveito da forma como as arestas e vértices são controlados na nossa estrutura. Ao explorar as arestas de saída através das funções da TAD, o algoritmo consegue iterar sobre os vizinhos, atualizando gradativamente as estimativas de distância até que os caminhos ótimos finais sejam estabelecidos.

Programas principais

Para testar a TAD do grafo e os algoritmos, foram desenvolvidos dois programas principais: **main** e o **cricaoGrafos**.

cricaoGrafos: Com ele é possível criar os grafos e salvar em formato de arquivo. Há também uma opção para carregar um grafo em texto e depois convertê-lo para *.dot*, tanto como digrafo ou grafo não-direcionado. O intuito aqui foi facilitar a criação de instâncias de teste.

main: Programa principal para utilizar os algoritmos de grafos. É necessário carregar o grafo a partir de um arquivo texto, não sendo possível a criação do mesmo nele. Uma vez carregado, é apresentado um menu para que o usuário escolha qual dos algoritmos utilizar naquele grafo. Quando um vetor de pesos é necessário para os algoritmos, como o *CMC* (caminho mais curto) e o *AGM* (árvore geradora mínima), foi desenvolvido uma função auxiliar para permitir a criação e preenchimento de um durante a execução do programa, sendo também possível carregar um vetor de pesos a partir de um arquivo texto.

Considerações finais

O trabalho nos demandou um certo tempo, porém seu desenvolvimento foi feito com certa constância e sem muita dificuldades. Foi muito interessante pensar nas implementações de grafos e seus respectivos algoritmos, o que nos ajudou na concepção e compreensão dessa ferramenta da computação. Alguns trechos do código foram sim gerados por **inteligência artificial generativa**, sobretudo nas partes de manipulação de arquivos.

Algumas fontes utilizadas:

paa_03Grafos (Slide da aula do Prof. Antonio Alfredo Ferreira Loureiro)

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to algorithms. 3. ed. Cambridge: MIT Press, 2009.

BACKES, André Ricardo. Algoritmos e estruturas de dados em linguagem C. 1. ed., 2. reimpr. Rio de Janeiro: LTC, 2025.